



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2283 — Diseño y Análisis de Algoritmos 2025-2

Interrogación 2

28 de octubre de 2025

Evaluación: 2 preguntas, cada una con subpreguntas. Cada pregunta vale 6 puntos (+1 punto base). La nota es el promedio de las 2 preguntas.

Forma de Entrega: Además de entregar las hojas con sus respuestas, debe subir una versión digital a Canvas. Al final de la prueba, tiene 10 minutos para subir la imagen de sus respuestas, en formato vertical, **cada pregunta por separado**. Es su responsabilidad asegurarse de que todas sus respuestas se suban correctamente al sistema.

Dispositivo: Para digitalizar la prueba se permite el uso de UN dispositivo (teléfono, tablet) el cual debe estar guardado y en silencio durante el desarrollo.

Torpedo: Se permite una hoja tamaño oficio escrita a mano por usted, usando ambos lados. También debe subirse una versión digital a Canvas.

Política de Salidas: Si necesita salir de la sala al baño, debe dejar el celular adelante en la mesa con los ayudantes.

Se sugiere leer todas las preguntas antes de iniciar las respuestas.

Tiempo: 2 horas.

1. Transformada Rápida de Fourier

1. El siguiente es el algoritmo estudiado en clases para calcular la transformada rápida de Fourier:

```
FFT( $\bar{a}$ )
1.  ( $a_0, \dots, a_{n-1}$ ) :=  $\bar{a}$ 
2.  if  $n = 2$  then
3.       $y_0 = a_0 + a_1$ 
4.       $y_1 = a_0 - a_1$ 
5.      return [ $y_0, y_1$ ]
6.  else
7.       $\bar{a}_0 := (a_0, \dots, a_{n/2-1})$ 
8.       $\bar{a}_1 := (a_{n/2}, \dots, a_{n-1})$ 
9.      [ $y_{0,0}, \dots, y_{0,n/2-1}$ ] := FFT( $\bar{a}_0$ )
10.     [ $y_{1,0}, \dots, y_{1,n/2-1}$ ] := FFT( $\bar{a}_1$ )
11.      $\omega_n := e^{\frac{2\pi i}{n}}$ 
12.      $\alpha := 1$ 
13.     for  $k := 0$  to  $\frac{n}{2} - 1$  do
14.          $y_k := y_{0,k} + \alpha \cdot y_{1,k}$ 
15.          $y_{\frac{n}{2}+k} := y_{0,k} - \alpha \cdot y_{1,k}$ 
16.          $\alpha := \alpha \cdot \omega_n$ 
17.     return [ $y_0, \dots, y_{n-1}$ ]
```

Responda los siguientes puntos.

- a) [1.0 punto] Indique qué recibe como entrada y qué retorna FFT.

Solución: Recibe como entrada un polinomio $p(x) = \sum_{i=0}^{n-1} a_i x^i$ representado en forma canónica $(a_0, \dots, a_{n-1})$. Produce como resultado el arreglo $[y_0, \dots, y_{n-1}]$ tal que $y_k = p(\omega_n^k)$ para $0 \leq k \leq n-1$.

Rúbrica: 1.0 puntos si la respuesta es correcta (0.5 puntos por la entrada correcta, 0.5 puntos por la salida correcta), puntaje parcial si no es del todo correcta. Es importante que indiquen qué son los y_k , sino considerar incorrecta y evaluar con 0 puntos.

- b) [1.0 punto] Explique brevemente los pasos del algoritmo FFT. Puede usar los números de línea para referenciar las partes del algoritmo.

Solución: Los principales pasos son el caso base (líneas 2–5), que corresponde a un polinomio de grado 1 $p(x) = a_0 + a_1 x$ que debe ser evaluado en las dos raíces de la unidad ω_2^0 y ω_2^1 (esas evaluaciones se retornan en y_1 y y_2). Las líneas 7 y 8 corresponden a la representación de los polinomios $p_e(x)$ y $p_o(x)$ vistos en clase. Las líneas 9 y 10 es la etapa de la evaluación recursiva de los polinomios mencionados antes en las $n/2$ raíces de la unidad. Luego la iteración de las líneas 13–16

es la etapa que combina los resultados obtenidos recursivamente para formar la solución. Aquí deben explicar brevemente el sentido de las líneas 14 y 15.

Rúbrica: puntaje completo si consideran todos esos pasos y los explican brevemente. Puntaje parcial si omiten alguno de esos pasos.

- c) [1.0 punto] Explique por qué el algoritmo FFT es correcto. No es necesaria una demostración matemática, sea breve e indique cuáles son las ideas centrales que muestran que el algoritmo es correcto.

Solución: el algoritmo lleva a cabo una estrategia dividir y conquistar, en donde el polinomio de entrada es evaluado en las n raíces de la unidad mediante la división de ese polinomio en 2 polinomios cuyo grado es la mitad del grado del polinomio original. Esos polinomios son evaluados en las n raíces elevadas al cuadrado, las cuales se transforman en las $n/2$ raíces de la unidad (reduciéndose en cantidad a la mitad).

Rúbrica: puntaje completo si consideran al menos todos esos puntos. Puntaje parcial en otro caso.

2. [3.0 puntos] Considere una base de datos que tiene dos tablas, A y B, cada una con una columna llamada **value** (entre otras columnas, pero ésta será la única de interés para nosotros). Cada fila en estas tablas asocia a **value** un valor en el intervalo entero $[0, n - 1]$. Un motor de base de datos debe ejecutar la siguiente consulta, que es computacionalmente costosa:

SELECT * FROM A, B WHERE A.value + B.value = t;

la cual busca todas las combinaciones de filas de las tablas A y B tal que la suma de sus campos **value** sea exactamente **t** (es decir, **A.value + B.value = t**). Por razones de eficiencia, antes de ejecutar la consulta, el motor debe *estimar cuántas filas* satisfacen esta condición. El motor procederá con la ejecución de dicha consulta si el número estimado de resultados es menor o igual a un umbral r . De otra manera, tendrá que seleccionar un plan de ejecución alternativo.

Para realizar esta estimación, el motor dispone del histograma $H_A = \{(0, f_0^{(A)}), \dots, (n-1, f_{n-1}^{(A)})\}$, tal que $f_i^{(A)}$ indica la cantidad de veces que el valor i ocurre en la columna **value** de A. Similarmente, se tiene $H_B = \{(0, f_0^{(B)}), \dots, (n-1, f_{n-1}^{(B)})\}$ para la tabla B.

Diseñe un algoritmo, usando pseudocódigo o en el lenguaje de programación de su preferencia, que reciba los histogramas H_A y H_B correspondientes a las tablas, y determine si el motor de base de datos debe ejecutar la consulta anterior o no. El algoritmo además recibe como entrada el valor objetivo **t** y el umbral máximo r . Puede suponer que n es una potencia de 2 y que dispone de las operaciones de *Transformada Rápida de Fourier* (FFT) y su inversa (IFFT).

Ejemplo: Sean $t = 4$, $r = 10$, $H_A = \{(0, 10), (1, 4), (2, 4)\}$ y $H_B = \{(0, 5), (1, 3), (2, 3)\}$. La cantidad total de filas que satisfacen la condición son 12, que son las combinaciones entre las 4 filas de A con las 3 filas B con **value** = 2. Por lo tanto, el motor no debe ejecutar la consulta.

Solución:

El problema es similar al ejercicio 5 de la guía. La idea es definir dos polinomios $p_A(x)$ y $p_B(x)$ de la siguiente forma **(punto (I) de la rúbrica)**:

$$p_A(x) = \sum_{(i, f_i^{(A)}) \in H_A} f_i^{(A)} \cdot x^i,$$

y

$$p_B(x) = \sum_{(i, f_i^{(B)}) \in H_B} f_i^{(B)} \cdot x^i.$$

Es correcto también si se utilizaron los índices $i \in \{1, 2, \dots, n - 1\}$.

Resolveremos el problema calculando $p_A(x) \cdot p_B(x)$. Note que, cuando multiplicamos ambos polinomios los exponentes se suman (representa la suma de los valores) y por lo tanto deberemos fijarnos en el coeficiente del exponente con valor t , ya que este

tendrá la cantidad de filas que suman t (**Punto (V) de la rúbrica**). Por lo tanto, podemos usar la FFT para transformar ambos polinomios de representación canónica a punto-valor(**Punto (II) de la rúbrica**), y luego multiplicar los dos polinomios (en la representación punto-valor)(**Punto (III) de la rúbrica**). Cuando transformamos el resultado de la multiplicación a la representación canónica usando la IFFT (**Punto (IV) de la rúbrica**) solo debemos fijarnos en la posición t -ésima para encontrar el coeficiente que representa a la cantidad de filas que cumplen la condición (**Puntos (V) y (VI) de la rúbrica**). El algoritmo quedaría de la siguiente forma:

Algoritmo 1: EJECUTARCONSULTA(H_A, H_B, t, k)

```

1 Punto (I) de la rúbrica
2 Sea  $a[0..2n-1] = (0, \dots, 0)$  el arreglo de coeficientes de  $p_A(x)$ 
3 Sea  $b[0..2n-1] = (0, \dots, 0)$  el arreglo de coeficientes de  $p_B(x)$ 
4 for  $(i, f_i^{(A)}) \in H_A$  do
5    $a[i] \leftarrow f_i^{(A)}$ 
6 for  $(i, f_i^{(B)}) \in H_B$  do
7    $b[i] \leftarrow f_i^{(B)}$ 
8 Punto (II) de la rúbrica
9  $V_a[0..2n-1] \leftarrow \text{FFT}(a)$ 
10  $V_b[0..2n-1] \leftarrow \text{FFT}(b)$ 
11 Punto (III) de la rúbrica
12 for  $k \leftarrow 0$  to  $2n-1$  do
13    $V_c[k] \leftarrow V_a[k] \cdot V_b[k]$ 
14 Punto (IV) de la rúbrica
15  $c[0..2n-1] \leftarrow \text{IFFT}(V_c)$ 
16 return  $c[t] \leq r$  Puntos (V) y (VI) de la rúbrica
```

Rúbrica:

Se evalúa la correcta aplicación y utilización de la FFT para la resolución problema, por lo que cualquier algoritmo propuesto debe tener las siguientes partes:

- (I) Correcta definición de los polinomios $p_A(x)$, $p_B(x)$, y su respectiva representación canónica del problema: **0.6 pts**
- (II) Transformación a punto-valor de ambos polinomios $p_A(x)$ y $p_B(x)$ usando la FFT: **0.8 pts**
- (III) Multiplicación en representación punto-valor de ambos polinomios en tiempo $\Theta(n)$: **0.5 pts**
- (IV) Transformación del polinomio resultante de la multiplicación de punto-valor a canónica usando la IFFT: **0.5 pts**
 - Cualquier asunción de incluir o no el factor $\frac{1}{n}$ en la IFFT está bien.

- (V) Identifica que el coeficiente en la posición t del resultado de $p_A(x) \cdot p_B(x)$ es la cantidad de filas que suman t : **0.5 pts**
- (VI) Verifica que la cantidad de filas es menor o igual a r : **0.1 pts**

2. Algoritmos Codiciosos

Dados $m \geq 2$ arreglos ordenados $A_1, A_2, \dots, A_m$ de tamaños $n_1, n_2, \dots, n_m$, y sea $n = \sum_{i=1}^m n_i$. Alice desea construir un único arreglo ordenado $A[1..n]$ que contenga todos los elementos de los arreglos dados. Para ello, dispone únicamente de una operación llamada $\text{MERGE}(B, C)$, la cual recibe dos arreglos ordenados B y C (de tamaños b y c elementos, respetivamente) y devuelve un nuevo arreglo ordenado de tamaño $b + c$ que contiene todos los elementos de B y C . En ese caso, el costo de ejecutar $\text{MERGE}(B, C)$ es $b + c$. El problema a resolver es el siguiente: dada una lista $L = \langle n_1, \dots, n_m \rangle$ con los tamaños de los arreglos $A_1, \dots, A_m$, encontrar una estrategia óptima para aplicar la operación MERGE repetidamente (combinando de a pares los arreglos disponibles) hasta obtener el arreglo final A , de modo que el costo total sea el mínimo posible.

Responda los siguientes puntos.

1. [1.5 puntos] Indique cómo modelaría una estrategia de combinación de los m arreglos, representándola con un árbol binario que llamaremos *árbol de combinación*. En dicho árbol, las m hojas corresponden a los arreglos de entrada $A_1, \dots, A_m$, mientras que los nodos internos corresponden al resultado de hacer MERGE a los arreglos representados por sus hijos.

- (I) Complete los detalles necesarios en la definición de estos árboles, asociando los valores enteros que considere necesarios a los nodos del árbol.

Solución: dado que cada nodo del árbol representa un arreglo involucrado en el proceso de Merge, cada nodo del árbol almacena un entero correspondiente al tamaño de dicho arreglo. En las hojas, se tiene el tamaño de los arreglos de entrada. En los nodos internos se tiene el tamaño del arreglo resultante del Merge correspondiente.

Rúbrica:

- Lo importante aquí es que agreguen a cada nodo el tamaño del arreglo que representa ese nodo. En las hojas, es el tamaño de los arreglos de entrada. En los nodos internos, es el tamaño del arreglo resultante de hacer Merge. Valorar también si agregan otro tipo de información que les sea útil para el proceso de entender el costo de una estrategia.

- (II) Muestre 2 árboles de combinación distintos para $L = \langle 6, 3, 2, 5 \rangle$. En este punto, no es necesario que los ejemplos que muestre correspondan a la estrategia óptima.

Rúbrica:

- Los árboles mostrados deben ser coherentes con la definición del punto anterior.

- (III) ¿Cuál es el costo total de cada una de las estrategias del punto anterior? Recuerde que aplicar MERGE tiene un costo asociado.

Solución: si en los árboles almacenan el tamaño de los arreglos correspondientes, entonces el costo es igual a la suma de los valores en los nodos internos del árbol. Las hojas corresponden a los arreglos de entrada y por lo tanto no se obtienen usando Merge.

Rúbrica:

- Aquí el costo debe estar correcto. Valorar intentos que no sean correctos pero se acerquen. Los casos en que respondan 16 (el tamaño del arreglo final) son valorados con 0 puntos.

2. [1.5 puntos] Defina formalmente la función objetivo a optimizar.

Hint: trate de usar lo aprendido en el punto anterior.

Solución: Para un árbol de combinación T como el definido en el punto anterior para una estrategia de combinación particular, sea $C(T)$ el costo total correspondiente de aplicar MERGE como lo indica T . Note que en T , un n_i cualquiera aparece sumando en todos los nodos internos que son ancestros de la hoja correspondiente a n_i . Eso significa que n_i aporta con $n_i \cdot \text{prof}(n_i)$ a $C(T)$, siendo $\text{prof}(n_i)$ la profundidad de la hoja de T correspondiente a n_i . Por lo tanto, el costo total es

$$C(T) = \sum_{i=1}^m n_i \cdot \text{prof}(n_i).$$

Esta es la función objetivo a minimizar.

Rúbrica:

- La definición de la función debe ser lo más formal posible. Si sólo lo indican con palabras, asignar puntaje parcial.
- Tener en cuenta que podrían definir la función objetivo como la suma de los valores en los nodos internos del árbol correspondiente. Una definición de ese tipo debe considerarse correcta.

3. [1.5 puntos] Escriba el algoritmo codicioso OPTIMALMERGE que permita obtener la estrategia de costo óptimo para mezclar los m arreglos mediante la aplicación de la función MERGE.

Solución: La estrategia codiciosa es similar a Huffman: en cada paso, el algoritmo toma los dos arreglos de menor longitud n_i y n_j , los combina usando MERGE con costo $n_i + n_j$, y reemplaza los dos arreglos por el resultado de su combinación. Se procede ahora recursivamente, habiendo reducido la cantidad de arreglos en 1. Cuando quedan solo 2 arreglos, simplemente se combinan y finaliza el proceso. El siguiente algoritmo expresa esa idea, entregando el costo total de la estrategia óptima (una alternativa era devolver la codificación de la estrategia usando el árbol de combinación correspondiente). Inicialmente debe ser invocado con el arreglo $L[1..m]$ que contiene los m tamaños de

los arreglos. Notar que, de acuerdo a la definición del problema en el enunciado, los arreglos $A_1, \dots, A_m$ no eran dados como entrada al algoritmo, sino que sólo sus tamaños.

Algoritmo 2: OPTIMALMERGE($L[1..j]$)

```

1 if  $j = 2$  then
2   | return  $L[1] + L[2]$ 
3 else
4   |  $\ell_1 \leftarrow \text{MIN}(L)$ 
5   |  $\text{DELETETEMIN}(L)$ 
6   |  $\ell_2 \leftarrow \text{MIN}(L)$ 
7   |  $\text{DELETETEMIN}(L)$ 
8   |  $\text{INSERT}(L, \ell_1 + \ell_2)$ 
9   | return  $\ell_1 + \ell_2 + \text{OPTIMALMERGE}(L[1..j - 1])$ 

```

Rúbrica: Tener en cuenta que lo importante es el proceso codicioso, no tener en cuenta la manera en que representan la solución (podrían devolver el costo óptimo, como lo hace el algoritmo anterior, o devolver el árbol que codifica la estrategia, entre otras posibilidades).

- **Propone un enfoque codicioso: 1.0 puntos** si el algoritmo propuesto usa una estrategia codiciosa óptima. Si la estrategia no es la óptima, asignar como máximo 0.5 puntos (el puntaje asignado depende de qué tanto se acerca la estrategia propuesta al óptimo).
- **Correctitud de la solución propuesta: 0.5 puntos** si la solución propuesta está implementada correctamente (aún cuando no sea la que produce la solución óptima). Por ejemplo, que ordenen bien la estructura L , para que puedan eliminar los dos mínimos y agregar la suma. Idealmente debería ser un heap de mínimos. Si lo hace vía ordenamiento y no tiene en cuenta esto, descontar **0.2 puntos**. Tener en cuenta consideraciones similares para otro tipo de soluciones.

4. [1.5 puntos] Demuestre que su estrategia minimiza el costo total para obtener A .

Solución: Sin pérdida de generalidad, asumimos que $n_1 \leq n_2 \leq \dots \leq n_m$. Dada la definición de $C(T)$, en una estrategia óptima T se debe cumplir:

- Si $n_i \leq n_j$, entonces $\text{prof}(n_i) \geq \text{prof}(n_j)$. Eso implica que

$$\text{prof}(n_1) \geq \text{prof}(n_2) \geq \dots \geq \text{prof}(n_m).$$

- $\text{prof}(n_1) = \text{prof}(n_2)$. Por la propiedad anterior, n_1 y n_2 son los nodos de mayor profundidad. Eso implica que si la profundidad de n_1 es mayor estricta que la de n_2 , entonces la profundidad de n_1 se puede acortar ya que el camino que va desde la raíz a n_1 en T es unario a partir de $\text{prof}(n_2)$. Así, se podría acortar la profundidad de n_1 , reduciendo $C(T)$ y contradiciendo el hecho de que es óptimo.

- En T , los dos arreglos más cortos, de largos n_1 y n_2 , deben haber sido combinados directamente usando MERGE, y por lo tanto son hojas hermanas en T . Si, por el contrario, no lo fueran, se podría construir una estrategia T' de la siguiente manera. Supongamos que la estrategia T hace MERGE(A_1, A_i) y MERGE(A_2, A_j), produciendo nodos internos de valores n_1+n_i y n_2+n_j . Note que si intercambiamos para hacer en su lugar MERGE(A_1, A_2) y MERGE(A_i, A_j), ahora los respectivos nodos internos corresponden a $n_1 + n_2$ y $n_i + n_j$. Por la propiedad anterior, n_1 y n_2 tienen la misma profundidad que n_i y n_j , por lo que no se modifica $C(T)$.

Demostremos a continuación por inducción que OPTIMALMERGE entrega el costo de la estrategia óptima de combinación:

Caso Base: Para $n = 2$, OPTIMALMERGE(L) entrega la suma de los largos de los arreglos más cortos, lo cual es óptimo (ya que no hay otra posibilidad).

Hipótesis Inductiva: Asumimos que OPTIMALMERGE(L) entrega el resultado óptimo para $|L| \in \{2, \dots, n-1\}$.

Paso Inductivo: Sea $|L| = n \geq 3$. Asumamos, por contradicción, que OPTIMALMERGE(L) produce una estrategia T que no es óptima. Eso significa que existe otra estrategia T^* óptima tal que

$$C(T^*) < C(T).$$

En T^* , n_1 y n_2 son hojas hermanas, al igual que en T . Consideremos la estrategia correspondiente a $L' = L - \{n_1, n_2\} \cup \{n_1 + n_2\}$. Note que, por HI, si consideramos el árbol T' resultante de invocar OPTIMALMERGE(L') en el algoritmo, tenemos que $C(T')$ es óptimo. Similarmente, sea $C(T^{*'})$ el costo que obtiene el algoritmo óptimo para el árbol $T^{*'}$ formado a partir de T^* y excluyendo las hojas de n_1 y n_2 . Entonces estamos diciendo que:

$$n_1 + n_2 + C(T^{*'}) < n_1 + n_2 + C(T'),$$

lo que es lo mismo que

$$C(T^{*'}) < C(T'),$$

lo cual no puede ser posible ya que por HI, $C(T')$ es óptimo.

Rúbrica:

- Si la demostración no es del todo formal pero muestran la intuición de la misma, asignar puntaje parcial (dependiendo de qué tan correcto sea lo que responden).